

SLOT 13

CHAPTER 10: RESTFUL WEB SERVICES WITH SPRING BOOT PART B – ANNOTATIONS + 3-LAYER CRUD

HTTP Methods • Request/Response Handling • Exception Handling • Postman

CLO3 | JDK 21 | Spring Boot 3.x | Project-Based Learning

Phạm vi slide chuẩn: Chapter 10, trang 18–36

Thông tin nhanh

Thành phần	Nội dung
Slot	13
Chủ đề	Chapter 10 – RESTful Web Services with Spring Boot, Part B
Trọng tâm	Annotations, 3-layer CRUD, HTTP Methods, Request/Response Handling
Môi trường	JDK 21, IntelliJ IDEA, Maven, Spring Boot 3.x, Spring Web, Lombok, Postman
Dữ liệu demo	In-memory Map/List; chưa dùng JPA/Database
Case xuyên suốt	FUNewsManagementSystem – resource News
Đầu ra tối thiểu	CRUD API chạy được + 3-layer + status đúng + exception handling + Postman evidence

Cách sử dụng giáo trình

- Giảng viên: dạy theo nhịp Explain → Trace → Build → Break → Verify → Transfer; tập trung vào luồng request đi qua Controller → Service → Repository.
- Sinh viên: không chỉ nhớ annotation; phải giải thích annotation đọc phần nào của HTTP request và ảnh hưởng phần nào của HTTP response.
- Demo 1 giúp hiểu endpoint tối thiểu; Demo 2 chuyển sang CRUD in-memory theo 3-layer để chuẩn bị Lab 03.
- AI có thể hỗ trợ giải thích/debug, nhưng mọi thay đổi phải được kiểm chứng bằng Postman, console/log và khả năng giải thích code.

MỤC LỤC NỘI DUNG

- 0. Sinh viên cần chuẩn bị gì trước khi học Slot 13
- I. Tóm tắt các chủ đề & mục tiêu trong bài học
- II. Trọng tâm cốt lõi & kỹ năng sinh viên cần nắm vững
- III. Ứng dụng thực tiễn trong sản phẩm
- IV. Hướng dẫn thực hiện và triển khai – Demo 1 + Demo 2
- V. Kinh nghiệm thực tiễn, tips & debugging
- VI. Bài tập chủ đề và bài tập tổng hợp
- VII. Foundation Glossary
- VIII. Research Base & AI Base Learning
- IX. Tự kiểm tra sau buổi học
- Phụ lục A. Quick Reference annotations
- Phụ lục B. Postman Verification Matrix
- Phụ lục C. Human Verification Gate & Evidence Package

0. SINH VIÊN CẦN CHUẨN BỊ GÌ TRƯỚC KHI HỌC SLOT 13

0.1. Kiến thức bắt buộc từ Slot 12 Part A

Kiến thức	Sinh viên phải nhớ	Câu hỏi kiểm tra nhanh
REST / RESTful API	Resource là trung tâm; URI định danh resource; method biểu diễn hành động.	/api/news là resource hay action?
HTTP Methods	GET đọc, POST tạo, PUT cập nhật toàn bộ, DELETE xóa.	POST tạo thành công nên trả status nào?
HTTP Status	200, 201, 400, 401, 403, 404, 500.	401 khác 403 ở điểm nào?
JSON	Representation phổ biến giữa frontend và backend.	JSON request body đi vào Java object bằng cơ chế nào?
Spring Boot overview	Starter, auto-configuration, embedded server.	Spring Web đem lại nhóm chức năng nào?

0.2. Java nền tảng cần ôn

- Class/object/method và constructor.
- Interface/implementation và dependency injection ở mức khái niệm.
- List, Map, Optional, Stream cơ bản.
- Exception: checked vs unchecked ở mức nhận biết.
- Java annotation là metadata, không phải “hàm tự chạy”.
- Maven: pom.xml, dependency, lifecycle cơ bản.

0.3. Công cụ và môi trường

Công cụ	Yêu cầu	Kiểm tra
JDK	21 theo baseline môn học	java -version
IntelliJ IDEA	Mở/chạy Maven Spring Boot project	IDE nhận diện pom.xml
Maven / Maven Wrapper	Build/run dependency	./mvnw -v hoặc mvnw.cmd -v
Spring Initializr	Tạo skeleton project	start.spring.io
Postman	Gửi GET/POST/PUT/DELETE	Tạo collection Slot13
Git	Checkpoint theo milestone	git status

Compatibility note: Spring Getting Started hiện yêu cầu Java 17 trở lên; JDK 21 của môn học phù hợp. Khi tạo project, chọn một Spring Boot 3.x phù hợp với môi trường lớp và tránh đổi major version giữa chừng.

0.4. Bài chuẩn bị trước lớp – 3-2-1

- 3 khái niệm: @RestController, @RequestBody, 3-layer architecture.
- 2 điều chưa rõ: ghi lại trước giờ học.
- 1 dự đoán: khi POST JSON tới /api/news, dữ liệu sẽ đi qua những lớp nào?

0.5. Pre-class Human Check

- POST dùng để làm gì?
- 201 Created dùng khi nào?
- Controller có nên trực tiếp lưu dữ liệu không? Vì sao?
- Path parameter và query parameter khác nhau về ý nghĩa?

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

I.1. Từ Part A sang Part B

Chủ đề Part A	Đã biết	Part B phải làm được
---------------	---------	----------------------

REST fundamentals	Biết resource/URI/representation	Biến API contract thành controller code
HTTP Methods	Biết CRUD ↔ method	Map method bằng @GetMapping/@PostMapping/...
Status Codes	Biết semantics	Trả status bằng @ResponseStatus/ResponseEntity
Request/Response	Biết cấu trúc HTTP	Bind path/query/body vào Java và serialize response
Spring Boot overview	Biết vai trò	Tạo/run project, tổ chức 3-layer, test bằng Postman

I.2. Định vị Slot 13 trong chuỗi backend

Slot	Chủ đề	Vai trò của Slot 13
12	REST fundamentals	Thiết kế trước khi code.
13	Annotations + 3-layer CRUD	Chuyển API contract thành code chạy được.
14	Documenting REST Services	Tài liệu hóa API bằng OpenAPI/Swagger.
15	Versioning, paging, sorting	Mở rộng khả năng API.
16	Testing / MockMvc + Lab 03	Kiểm thử tự động endpoint.
17	JPA / Entity / Repository	Thay in-memory repository bằng persistence thật.
18	REST Controller + Lab 04	Kết nối đầy đủ với tầng dữ liệu.

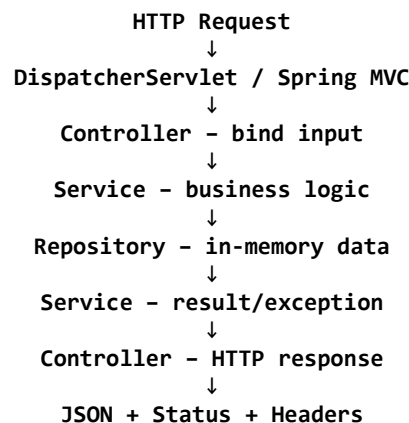
I.3. Bối cảnh

Assignment 01 mới dừng ở frontend ReactJS và mock data. Slot 13 bắt đầu biến “hộp đồng API” của FUNewsManagementSystem thành backend thực thi được. Trọng tâm không phải viết thật nhiều annotation, mà là hiểu một HTTP request được Spring MVC ánh xạ, bind dữ liệu, đi qua các layer, rồi tạo HTTP response như thế nào.

I.4. Learning outcomes của buổi học

- Tạo Spring Boot project có Spring Web và Lombok bằng Spring Initializr.
- Giải thích và sử dụng @RestController, @RequestMapping và các mapping annotation.
- Phân biệt @PathVariable, @RequestParam, @RequestBody và @ResponseBody.
- Triển khai CRUD in-memory đúng luồng Controller → Service → Repository.
- Trả status code phù hợp bằng @ResponseStatus hoặc ResponseEntity.
- Xử lý resource not found bằng @ExceptionHandler + @ControllerAdvice.
- Kiểm thử GET/POST/PUT/DELETE bằng Postman và đọc request/response.
- Giải thích idempotency của GET/PUT/DELETE và lý do POST không idempotent theo semantics thông thường.
- Chuẩn bị source/evidence cho Lab 03.

I.5. Mental model trung tâm



I.6. 135 phút gợi ý trên lớp

Thời lượng	Hoạt động	Sản phẩm
0–15	Retrieval Slot 12 + predict request flow	Bảng mapping HTTP
15–35	Annotations core + mini traces	Annotation quick sheet
35–55	Demo 1 Spring Initializr + GET	Endpoint /api/hello chạy
55–85	Demo 2 3-layer CRUD – build cùng giảng viên	GET/POST/PUT/DELETE
85–105	Request/Response + exception handling	404/201/204 đúng
105–120	Break-It & Postman verification	Debug log
120–135	FUNews transfer task + Exit Ticket	API mini-design + evidence

I.7. Tài liệu tham khảo

- Slide Chapter 10, trang 18–36 – nguồn chính theo syllabus.
- Spring: Building a RESTful Web Service – Spring Getting Started.
- Spring Framework Reference – annotated controllers, @RequestBody, ResponseEntity, exception handling.
- Tài liệu Slot 12 – REST Fundamentals.

II. TRỌNG TÂM CỐT LỖI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

II.1. Bản đồ annotation: annotation đọc gì / tạo gì?

Annotation / API	Nằm ở đâu	Vai trò	Ví dụ
@RestController	Class	Controller REST; return value đi vào response body	NewsController
@RequestMapping	Class/Method	Base path hoặc mapping tổng quát	/api/news
@GetMapping	Method	Map HTTP GET	GET /api/news
@PostMapping	Method	Map HTTP POST	POST /api/news
@PutMapping	Method	Map HTTP PUT	PUT /api/news/{id}
@DeleteMapping	Method	Map HTTP DELETE	DELETE /api/news/{id}
@PathVariable	Parameter	Đọc URI template variable	{id} → Long id
@RequestParam	Parameter	Đọc query parameter	?keyword=spring
@RequestBody	Parameter	Deserialize HTTP body → Java object	JSON → News
@ResponseBody	Method/Class semantics	Serialize return value → body	Object → JSON
@ResponseStatus	Method/Exception	Gán status cố định	201 Created
@ExceptionHandler	Method	Xử lý loại exception cụ thể	NewsNotFoundException
@ControllerAdvice	Class	Áp dụng exception handling đa controller	GlobalExceptionHandler

Modern practice: Khi status/headers/body thay đổi theo từng kết quả, ResponseEntity thường rõ hơn @ResponseStatus cố định. Giáo trình sử dụng cả hai để sinh viên hiểu semantics HTTP.

II.2. @RestController và @ResponseBody

@RestController là stereotype dành cho REST controller; về mặt semantics, nó kết hợp @Controller và @ResponseBody. Do đó object được trả về từ handler method sẽ được ghi vào HTTP response body thông qua message converter (thường thành JSON khi Jackson có trên classpath).

```
@RestController
@RequestMapping("/api/news")
public class NewsController {
    // handler methods
}
```

Hiểu sai thường gặp: `@RestController` không tự tạo CRUD và không thay `Service/Repository`. Nó chỉ đánh dấu lớp xử lý request theo mô hình REST.

II.3. @RequestMapping và shortcut mappings

Nên dùng `@RequestMapping` ở cấp class cho base URI; ở cấp method ưu tiên

`@GetMapping/@PostMapping/@PutMapping/@DeleteMapping` để code phản ánh trực tiếp HTTP semantics.

```
@RestController
@RequestMapping("/api/news")
public class NewsController {

    @GetMapping
    public List<News> getAll() { ... }

    @GetMapping("/{id}")
    public News getOne(@PathVariable Long id) { ... }
}
```

II.4. @PathVariable vs @RequestParam vs @RequestBody

Nguồn dữ liệu	Annotation	Khi dùng	Ví dụ
Path	@PathVariable	Định danh resource hoặc thành phần bắt buộc của URI	/api/news/10
Query string	@RequestParam	Filter/search/sort/pagination/optional option	/api/news?keyword=spring
HTTP body	@RequestBody	Payload tạo/cập nhật	POST JSON News

```
@GetMapping("/{id}")
public News getOne(@PathVariable Long id) { ... }

@GetMapping
public List<News> search(@RequestParam(required = false) String keyword) { ... }

@PostMapping
public News create(@RequestBody News request) { ... }
```

II.5. Request/Response handling – đừng chỉ nhìn Java method

HTTP phần nào	Spring đọc/ghi bằng gì	Ví dụ
Method + URI	Mapping annotations	POST /api/news
Path variable	@PathVariable	id=10
Query param	@RequestParam	keyword=spring
Body JSON	@RequestBody + HttpMessageConverter	JSON → News
Response body	Return object / ResponseEntity	News → JSON
Status	@ResponseStatus / ResponseEntity	201, 204, 404
Headers	ResponseEntity / framework	Location, Content-Type

POST /api/news HTTP/1.1

Content-Type: application/json

```
{
  "title": "Spring Boot REST",
  "content": "Slot 13 demo",
  "categoryId": 1,
  "active": true
}
```

HTTP/1.1 201 Created

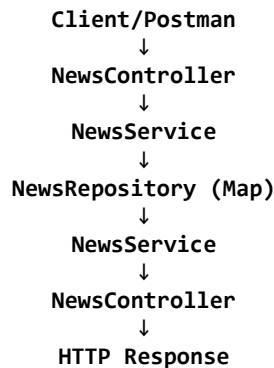
Location: /api/news/4

Content-Type: application/json

```
{
  "id": 4,
  "title": "Spring Boot REST",
  "content": "Slot 13 demo",
  "categoryId": 1,
  "active": true
}
```

II.6. Kiến trúc 3-layer

Layer	Trách nhiệm chính	Không nên làm
Controller	HTTP mapping, bind input, chọn response	Không chứa business logic dài, không tự quản lý storage
Service	Business rules, orchestration, validation nghiệp vụ	Không phụ thuộc chi tiết HTTP như URL/status khi không cần
Repository	Truy xuất/lưu dữ liệu	Không quyết định response status hoặc UI



Scope Slot 13: Repository dùng Map/ConcurrentHashMap để học architecture. JPA Repository và database thật thuộc chapter sau.

II.7. HTTP Methods và idempotency

Method	CRUD	Body	Status thường gặp	Idempotent?
GET	Read	Không	200 / 404	Có
POST	Create	Có	201 / 400	Không
PUT	Update/replace	Có	200 / 404	Có về semantics
DELETE	Delete	Thường không	204 / 404	Có về semantics

Idempotent nghĩa là gửi cùng request nhiều lần có cùng hiệu ứng mong muốn trên server như gửi một lần. Không đồng nghĩa response phải giống từng byte hoặc status luôn giống nhau.

II.8. Status code: từ “method chạy” sang “API đúng semantics”

Tình huống	Status khuyến nghị	Lý do
GET thành công	200 OK	Có representation trả về

POST tạo resource	201 Created	Resource mới được tạo
DELETE thành công, không body	204 No Content	Không cần representation
JSON/input không hợp lệ	400 Bad Request	Client gửi request không hợp lệ
Chưa xác thực	401 Unauthorized	Cần authentication
Đã xác thực nhưng cấm	403 Forbidden	Không đủ quyền
Không có resource	404 Not Found	ID/path hợp lệ nhưng resource không tồn tại
Lỗi server không dự kiến	500 Internal Server Error	Server thất bại khi xử lý

II.9. Exception handling

Thay vì mỗi controller tự try/catch và tạo response khác nhau, có thể centralize lỗi bằng `@ControllerAdvice`. Đây là bước quan trọng để response lỗi nhất quán.

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(NewsNotFoundException.class)
    public ResponseEntity<ApiError> handleNotFound(NewsNotFoundException ex) {
        ApiError error = new ApiError(404, "Not Found", ex.getMessage());
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }
}
```

II.10. Kỹ năng đầu ra bắt buộc

- Trace được một request từ Postman tới repository và quay về response.
- Đọc annotation và nói được nó bind dữ liệu từ path/query/body nào.
- Viết CRUD endpoints đúng HTTP semantics.
- Giải thích vì sao Controller không nên chứa business logic.
- Tạo 404 có cấu trúc khi id không tồn tại.
- Dùng Postman chứng minh 200/201/204/404.
- Sửa được một endpoint khi giảng viên đổi URI, field hoặc rule nhỏ.

III. ỨNG DỤNG THỰC TIỄN KIẾN THỨC TRONG SẢN PHẨM

III.1. Case study xuyên suốt – FUNewsManagementSystem

Frontend action	REST request	Controller input	Service intent	Expected response
Mở News page	GET /api/news	Không body	findAll()	200 + JSON array
Xem News id 5	GET /api/news/5	@PathVariable id	findById(5)	200 hoặc 404
Tạo News	POST /api/news	@RequestBody News	create(request)	201 + Location + JSON
Sửa News	PUT /api/news/5	id + body	update(5, request)	200 hoặc 404
Xóa News	DELETE /api/news/5	id	delete(5)	204 hoặc 404
Tìm News	GET /api/news?keyword=java	@RequestParam keyword	search(keyword)	200 + array

III.2. Vì sao 3-layer quan trọng trong hệ thống thật?

- Khi chuyển Map → JPA, Controller gần như không cần biết storage đã đổi.
- Business rule có thể được test riêng ở Service mà không cần HTTP.
- Repository có thể thay implementation mà Service API vẫn giữ ổn định.
- Frontend chỉ phụ thuộc API contract, không phụ thuộc Java class nội bộ.

III.3. Tình huống thực tế khác

Domain	Resource ví dụ	Rule cần Service layer
E-commerce	products, orders	Không tạo order nếu stock không đủ

Banking	accounts, transfers	Không cho số dư âm; transaction atomic
Learning	courses, enrollments	Không enroll khi lớp đầy
Social	posts, comments	Chỉ author/admin được xóa

III.4. Architecture transfer

Trong Slot 13, NewsRepository chỉ là in-memory. Khi học JPA, repository sẽ trở thành persistence layer thật; khi tích hợp React, frontend vẫn gọi các endpoint giống nhau nếu API contract được giữ ổn định. Đây là lý do cần tách HTTP concern, business logic và data access ngay từ đầu.

IV. HƯỚNG DẪN THỰC HIỆN VÀ TRIỂN KHAI

IV.0. Cấu trúc project mục tiêu

```

slot13-rest-api/
├── src/main/java/com/fpt/sba301/slot13/
│   ├── Slot13Application.java
│   ├── controller/
│   │   ├── HelloController.java
│   │   └── NewsController.java
│   ├── service/
│   │   └── NewsService.java
│   ├── repository/
│   │   └── NewsRepository.java
│   ├── model/
│   │   └── News.java
│   └── exception/
│       ├── NewsNotFoundException.java
│       ├── ApiError.java
│       └── GlobalExceptionHandler.java

```

Package rule: Đặt application class ở package gốc để component scanning tìm thấy controller/service/repository ở các package con.

IV.A. Annotation Lab – từng annotation phải trả lời 3 câu hỏi

Câu hỏi	Ý nghĩa
1. Annotation đặt ở đâu?	Class / method / parameter
2. Nó đọc hoặc cấu hình phần nào?	URI, method, query, body, response, exception
3. Nếu bỏ annotation thì hiện tượng gì?	404 mapping, body không bind, status sai, handler không chạy...

A1. @RestController + @RequestMapping

```

@RestController
@RequestMapping("/api/news")
public class NewsController {
}

```

`@RequestMapping("/api/news")` tạo base path dùng chung. Tất cả method mapping bên trong sẽ nối tiếp path này.

A2. @GetMapping / @PostMapping / @PutMapping / @DeleteMapping

```

@GetMapping
public List<News> getAll() { ... }

@PostMapping
public News create(@RequestBody News request) { ... }

@PutMapping("/{id}")
public News update(@PathVariable Long id, @RequestBody News request) { ... }

@DeleteMapping("/{id}")
public void delete(@PathVariable Long id) { ... }

```

A3. @PathVariable

```
@GetMapping("/{id}")
public News getOne(@PathVariable Long id) {
    return newsService.findById(id);
}
```

Spring lấy segment `{id}` từ URI và chuyển kiểu sang Long. Nếu tên parameter khác tên placeholder, khai báo rõ `@PathVariable("id") Long newsId`.

A4. @RequestParam

```
@GetMapping
public List<News> getAll(
    @RequestParam(required = false, defaultValue = "") String keyword) {
    return keyword.isBlank() ? newsService.findAll() : newsService.search(keyword);
}
```

Query param phù hợp với filter/search/sort/paging. Không dùng path `/api/news/search/java` khi bản chất `java` chỉ là tiêu chí tìm kiếm tùy chọn.

A5. @RequestBody / @ResponseBody

```
@PostMapping
public News create(@RequestBody News request) {
    return newsService.create(request);
}
```

`@RequestBody` yêu cầu Spring đọc HTTP body và chuyển JSON thành Java object thông qua message converter. Với `@RestController`, return value đã có semantics tương đương `@ResponseBody`; không cần lặp lại trên từng method.

A6. @ResponseStatus và ResponseEntity

```
@PostMapping("/simple")
@ResponseStatus(HttpStatus.CREATED)
public News createSimple(@RequestBody News request) {
    return newsService.create(request);
}

@PostMapping
public ResponseEntity<News> create(@RequestBody News request) {
    News created = newsService.create(request);
    URI location = URI.create("/api/news/" + created.getId());
    return ResponseEntity.created(location).body(created);
}
```

`@ResponseStatus` tốt khi status cố định. `ResponseEntity` phù hợp hơn khi muốn điều khiển status, headers (ví dụ Location) và body theo kết quả runtime.

A7. @ExceptionHandler + @ControllerAdvice

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(NewsNotFoundException.class)
    public ResponseEntity<ApiError> handleNotFound(NewsNotFoundException ex) {
        ApiError body = new ApiError(
            404,
            "Not Found",
            ex.getMessage()
        );
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(body);
    }
}
```

Professional note: `@RestControllerAdvice` là convenience annotation cho `@ControllerAdvice` + response-body semantics. Bản học dùng `@ControllerAdvice` để sinh viên nhìn rõ vai trò từng cơ chế.

IV.B. Accessing REST applications

Công cụ	Dùng khi nào	Điểm cần quan sát
Postman	Thực hành chính Slot 13	Method, URL, headers, JSON body, status, response body

Browser	GET đơn giản	Nhanh nhưng không thuận tiện cho POST/PUT/DELETE
curl	Terminal/automation	Request có thể lưu vào script
Swagger UI	Sau khi có OpenAPI	Slot 14 đi sâu
Insomnia	HTTP client thay thế	Tương đương Postman ở nhiều use case

IV.C. DEMO 1 – Spring Boot + IntelliJ + endpoint đầu tiên

Bước C1 – Tạo project

Spring Initializr field	Giá trị gợi ý
Project	Maven
Language	Java
Spring Boot	3.x phù hợp môi trường lớp
Group	com.fpt.sba301
Artifact	slot13-rest-api
Packaging	Jar
Java	21
Dependencies	Spring Web, Lombok

Bước C2 – Kiểm tra pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>
```

Bước C3 – Tạo HelloController

```
package com.fpt.sba301.slot13.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

@RestController
public class HelloController {

    @GetMapping("/api/hello")
    public Map<String, String> hello(
        @RequestParam(defaultValue = "Student") String name) {
        return Map.of(
            "message", "Hello " + name,
            "slot", "13"
        );
    }
}
```

Bước C4 – Run và verify

```
GET http://localhost:8080/api/hello
GET http://localhost:8080/api/hello?name=Phuc
```

Check	Expected
Application startup	Embedded server chạy; không có stack trace đỏ
GET /api/hello	200 OK + JSON
GET ?name=Phuc	message thay đổi
Content-Type	application/json

Checkpoint V1: Sinh viên phải giải thích được vì sao Map được trả thành JSON và `name` lấy từ query string.

IV.D. DEMO 2 – Simple CRUD RESTful Web Service theo 3-layer

D1. Model – News.java

```
package com.fpt.sba301.slot13.model;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class News {
    private Long id;
    private String title;
    private String content;
    private Long categoryId;
    private boolean active;
}
```

Lombok sinh getter/setter, constructor và các method tiện ích tại compile time. Nếu IDE báo đỏ, kiểm tra Lombok plugin/annotation processing; build Maven mới là bằng chứng cuối.

D2. Repository – NewsRepository.java (in-memory)

```

package com.fpt.sba301.slot13.repository;

import com.fpt.sba301.slot13.model.News;
import org.springframework.stereotype.Repository;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;

@Repository
public class NewsRepository {
    private final Map<Long, News> store = new ConcurrentHashMap<>();
    private final AtomicLong sequence = new AtomicLong(0);

    public NewsRepository() {
        save(new News(null, "Welcome to SBA301",
            "REST API demo", 1L, true));
        save(new News(null, "Spring Boot Part B",
            "Annotations and 3-layer CRUD", 1L, true));
    }

    public List<News> findAll() {
        return new ArrayList<>(store.values());
    }

    public Optional<News> findById(Long id) {
        return Optional.ofNullable(store.get(id));
    }

    public News save(News news) {
        if (news.getId() == null) {
            news.setId(sequence.incrementAndGet());
        }
        store.put(news.getId(), news);
        return news;
    }

    public News update(Long id, News news) {
        news.setId(id);
        store.put(id, news);
        return news;
    }

    public boolean deleteById(Long id) {
        return store.remove(id) != null;
    }

    public List<News> search(String keyword) {
        String normalized = keyword == null ? "" : keyword.trim().toLowerCase();
        return store.values().stream()
            .filter(n -> n.getTitle().toLowerCase().contains(normalized))
            .toList();
    }
}

```

Why Map?: Map giúp demo CRUD mà không kéo JPA/SQL vào Slot 13. Đây là data-access layer giả lập, không phải cách persistence production.

D3. Exception – NewsNotFoundException.java

```

package com.fpt.sba301.slot13.exception;

public class NewsNotFoundException extends RuntimeException {
    public NewsNotFoundException(Long id) {
        super("News not found with id = " + id);
    }
}

```

D4. Service – NewsService.java

```

package com.fpt.sba301.slot13.service;

import com.fpt.sba301.slot13.exception.NewsNotFoundException;
import com.fpt.sba301.slot13.model.News;
import com.fpt.sba301.slot13.repository.NewsRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
@RequiredArgsConstructor
public class NewsService {
    private final NewsRepository newsRepository;

    public List<News> findAll() {
        return newsRepository.findAll();
    }

    public News findById(Long id) {
        return newsRepository.findById(id)
            .orElseThrow(() -> new NewsNotFoundException(id));
    }

    public List<News> search(String keyword) {
        return newsRepository.search(keyword);
    }

    public News create(News request) {
        request.setId(null);
        return newsRepository.save(request);
    }

    public News update(Long id, News request) {
        findById(id);
        return newsRepository.update(id, request);
    }

    public void delete(Long id) {
        findById(id);
        newsRepository.deleteById(id);
    }
}

```

Service chịu trách nhiệm “resource có tồn tại không?” trước update/delete. Controller không cần biết storage là Map hay database.

D5. Controller – NewsController.java

```

package com.fpt.sba301.slot13.controller;

import com.fpt.sba301.slot13.model.News;
import com.fpt.sba301.slot13.service.NewsService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.net.URI;
import java.util.List;

@RestController
@RequestMapping("/api/news")
@RequiredArgsConstructor
public class NewsController {
    private final NewsService newsService;

    @GetMapping
    public List<News> getAll(
        @RequestParam(required = false) String keyword) {
        if (keyword == null || keyword.isBlank()) {
            return newsService.findAll();
        }
        return newsService.search(keyword);
    }

    @GetMapping("/{id}")
    public News getOne(@PathVariable Long id) {
        return newsService.findById(id);
    }

    @PostMapping
    public ResponseEntity<News> create(@RequestBody News request) {
        News created = newsService.create(request);
        URI location = URI.create("/api/news/" + created.getId());
        return ResponseEntity.created(location).body(created);
    }

    @PutMapping("/{id}")
    public ResponseEntity<News> update(
        @PathVariable Long id,
        @RequestBody News request) {
        return ResponseEntity.ok(newsService.update(id, request));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        newsService.delete(id);
        return ResponseEntity.noContent().build();
    }
}

```

D6. Error response – ApiError.java

```

package com.fpt.sba301.slot13.exception;

public record ApiError(
    int status,
    String error,
    String message
) { }

```

D7. GlobalExceptionHandler.java

```
package com.fpt.sba301.slot13.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;

@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(NewsNotFoundException.class)
    public ResponseEntity<ApiError> handleNotFound(NewsNotFoundException ex) {
        ApiError body = new ApiError(
            404,
            "Not Found",
            ex.getMessage()
        );
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(body);
    }
}
```

D8. Build / run

```
# Windows
mvnw.cmd spring-boot:run

# macOS/Linux
./mvnw spring-boot:run

# Hoặc chạy Slot13Application từ IntelliJ
```

D9. Postman test sequence

#	Request	Body	Expected
1	GET /api/news	—	200 + array
2	GET /api/news/1	—	200 + object
3	GET /api/news/999	—	404 + ApiError
4	POST /api/news	JSON hợp lệ	201 + Location + object
5	PUT /api/news/1	JSON cập nhật	200 + updated object
6	GET /api/news?keyword=spring	—	200 + filtered array
7	DELETE /api/news/1	—	204
8	GET /api/news/1	—	404

D10. JSON bodies dùng để test

```
// POST
{
  "title": "RESTful API with Spring Boot",
  "content": "Created from Postman",
  "categoryId": 2,
  "active": true
}

// PUT /api/news/2
{
  "title": "Spring Boot CRUD Updated",
  "content": "Updated by PUT",
  "categoryId": 1,
  "active": false
}
```

IV.E. Trace request xuyên 3-layer

Stage	GET /api/news/2	POST /api/news
HTTP	GET + URI	POST + Content-Type + JSON body
Controller	@PathVariable id=2	@RequestBody JSON → News
Service	findById(2)	create(request), reset id

Repository	store.get(2)	generate id + store.put
Return	News object	Created News
HTTP response	200 JSON hoặc 404	201 + Location + JSON

IV.F. Break-It Lab – học bằng lỗi

Bug injection	Triệu chứng dự kiến	Điểm cần kiểm tra
Xóa @RestController	Endpoint 404 / controller không được đăng ký đúng	Component scanning + annotation
Đổi @PostMapping thành @GetMapping	POST trả 405	Method mapping
Xóa @RequestBody	Object không bind như mong đợi	Request body binding
Sai path variable name	Binding/mapping error tùy khai báo	URI template vs parameter
Ném NewsNotFoundException nhưng xóa advice	500 thay vì 404 có cấu trúc	Exception mapping
Controller gọi Repository trực tiếp	Vẫn chạy nhưng vi phạm architecture	Layer responsibility

IV.G. Professional upgrade – DTO awareness

Demo sử dụng model trực tiếp để tập trung vào annotations. Trong production, request/response DTO giúp giảm coupling, kiểm soát field client được gửi/nhận và chuẩn bị validation. Đây là awareness, chưa bắt buộc trong bài core.

```
public record NewsRequest(
    String title,
    String content,
    Long categoryId,
    boolean active
) { }

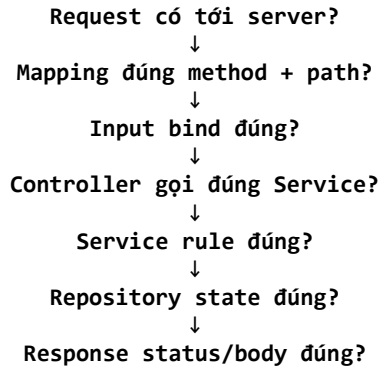
public record NewsResponse(
    Long id,
    String title,
    Long categoryId,
    boolean active
) { }
```

V. KINH NGHIỆM THỰC TIỄN, TIPS & DEBUGGING

V.1. 12 lỗi phổ biến

Lỗi	Dấu hiệu	Nguyên nhân gốc	Cách xử lý
404 cho mọi endpoint	Postman không vào controller	Sai base path/package scan/annotation	Kiểm tra @RestController, @RequestMapping, package
405 Method Not Allowed	URL đúng nhưng method sai	Mapping verb không khớp	So request với @Get/@Post/...
415 Unsupported Media Type	POST/PUT bị từ chối	Thiếu/sai Content-Type	Chọn raw JSON, application/json
400 khi bind body	Request lỗi	JSON sai/type mismatch	Đọc body và console
id null sau POST	Dữ liệu mới không có id	Repository chưa generate id	Kiểm tra save/sequence
PUT tạo item không tồn tại	Update sai semantics	Không check existence	Service findById trước update
DELETE 200 + body vô nghĩa	Semantics chưa tốt	Response design	Cần nhắc 204
Not found thành 500	Stack trace internal	Chưa map exception	@ControllerAdvice + @ExceptionHandler
Controller quá dài	Khó test/bảo trì	Business logic nằm sai layer	Move sang Service
State mất khi restart	Data biến mất	In-memory scope	Đúng trong Slot 13; JPA học sau
Lombok IDE báo đỏ	Getter/setter không nhận	Plugin/annotation processing	Build Maven để xác minh
Port 8080 occupied	App không start	Process khác dùng port	Dừng process hoặc đổi server.port

V.2. Debug decision tree



V.3. Best practices trong phạm vi Slot 13

- Dùng constructor injection; tránh field injection.
- Giữ Controller mỏng: mapping + bind + response; business logic ở Service.
- Trả 201 cho create, 204 cho delete khi không có body, 404 cho missing resource.
- Dùng @RequestParam cho filter/search; @PathVariable cho resource identity.
- Tạo error response nhất quán thay vì trả raw stack trace/message ngẫu nhiên.
- Không lẫn JPA vào buổi này; hiểu 3-layer trước rồi mới đổi repository implementation.
- Test endpoint bằng Postman ngay sau mỗi milestone, không chờ tới cuối.

V.4. Git checkpoints

Checkpoint	Nội dung commit gợi ý
V0	chore: create Spring Boot project
V1	feat: add hello REST endpoint
V2	feat: add News model and in-memory repository
V3	feat: add News service layer
V4	feat: add News CRUD controller
V5	feat: add global not-found handling
V6	test: verify CRUD with Postman

VI. BÀI TẬP CHỦ ĐỀ VÀ BÀI TẬP TỔNG HỢP

VI.A. Bài tập chủ đề

Bài 1: Annotation mapping

Viết 5 endpoint Product và chọn annotation đúng; giải thích input đến từ path/query/body.

- Evidence: screenshot/log/commit hoặc bảng trace theo yêu cầu giảng viên.

Bài 2: Path vs query

Thiết kế GET by id và search by keyword; giải thích vì sao mỗi tham số nằm ở vị trí đó.

- Evidence: screenshot/log/commit hoặc bảng trace theo yêu cầu giảng viên.

Bài 3: Status semantics

Cho 10 tình huống và chọn 200/201/204/400/401/403/404/500.

- Evidence: screenshot/log/commit hoặc bảng trace theo yêu cầu giảng viên.

Bài 4: 3-layer responsibility

Phân loại 15 câu lệnh/logic vào Controller, Service hoặc Repository.

- Evidence: screenshot/log/commit hoặc bảng trace theo yêu cầu giảng viên.

Bài 5: Request trace

Trace POST /api/news từ JSON tới Map và quay về response.

- Evidence: screenshot/log/commit hoặc bảng trace theo yêu cầu giảng viên.

Bài 6: Exception handling

Bổ sung CategoryNotFoundException + handler trả 404.

- Evidence: screenshot/log/commit hoặc bảng trace theo yêu cầu giảng viên.

Bài 7: Idempotency

Thử gọi PUT và DELETE lặp lại; mô tả state server và response có thể thay đổi.

- Evidence: screenshot/log/commit hoặc bảng trace theo yêu cầu giảng viên.

Bài 8: Break-It

Cố ý tạo 405, 415, 404 rồi ghi cách chẩn đoán.

- Evidence: screenshot/log/commit hoặc bảng trace theo yêu cầu giảng viên.

Bài 9: Postman

Tạo collection có biến baseUrl và 8 requests CRUD.

- Evidence: screenshot/log/commit hoặc bảng trace theo yêu cầu giảng viên.

Bài 10: Refactor

Cho controller chứa logic search; refactor logic sang Service.

- Evidence: screenshot/log/commit hoặc bảng trace theo yêu cầu giảng viên.

VI.B. Bài tập tổng hợp – Simple CRUD REST API for FUNewsManagementSystem

Mục tiêu: chọn một resource (ưu tiên News) và xây dựng REST API in-memory theo 3-layer, có exception handling và Postman evidence. Đây là cầu nối trực tiếp sang Lab 03.

Requirement	Minimum acceptance
Project	Spring Boot + Spring Web + Java 21 chạy được
Architecture	Controller → Service → Repository; không gọi chéo layer
GET list	200 + JSON array
GET by id	200 hoặc 404
POST	201 + resource mới; ưu tiên Location header
PUT	200 + resource cập nhật; 404 nếu id không có
DELETE	204; 404 nếu id không có
Search	GET collection với query param
Exception	@ControllerAdvice + @ExceptionHandler
Postman	Collection hoặc evidence đầy đủ

VI.C. Rubric tự kiểm tra trước Lab 03

Tiêu chí	Đạt khi
HTTP correctness	Method/path/status đúng semantics
Annotation understanding	Giải thích được từng annotation đang dùng
Architecture	Controller mỏng; Service xử lý rule; Repository quản lý data
CRUD behavior	Create/read/update/delete chạy đúng
Error handling	Missing id → 404 có body rõ
Verification	Có Postman evidence và biết đọc response
Explainability	Trace được ít nhất một GET và một POST end-to-end

VII. TỪ ĐIỂN THUẬT NGỮ – FOUNDATION GLOSSARY

Thuật ngữ	Định nghĩa	Ví dụ
-----------	------------	-------

@RestController	REST controller; response body semantics mặc định	class NewsController
@RequestMapping	Map base path hoặc request tổng quát	/api/news
@GetMapping	Map GET request	GET /api/news
@PostMapping	Map POST request	POST /api/news
@PutMapping	Map PUT request	PUT /api/news/1
@DeleteMapping	Map DELETE request	DELETE /api/news/1
@PathVariable	Bind URI template variable	{id} → Long id
@RequestParam	Bind query/form request parameter	?keyword=spring
@RequestBody	Deserialize request body → object	JSON → News
@ResponseBody	Serialize return value → response body	News → JSON
@ResponseStatus	Đặt status cố định cho handler/exception	201 Created
ResponseEntity	Bao status + headers + body	created(location).body(news)
@ExceptionHandler	Xử lý exception cụ thể	NewsNotFoundException
@ControllerAdvice	Cross-controller advice/exception handling	GlobalExceptionHandler
Controller	HTTP boundary layer	NewsController
Service	Business logic/orchestration	NewsService
Repository	Data access abstraction	NewsRepository
3-layer architecture	Controller–Service–Repository	request flow
DTO	Object chuyên chở data qua boundary	NewsRequest
Lombok	Giảm boilerplate Java	@Data, @RequiredArgsConstructor
Spring Initializr	Tạo skeleton Spring project	start.spring.io
Spring Web	Starter cho Spring MVC/REST	spring-boot-starter-web
Postman	HTTP API client	test CRUD
HTTP Method	Verb mô tả semantics request	GET/POST/PUT/DELETE
Request	Thông điệp client → server	method+URI+headers+body
Response	Thông điệp server → client	status+headers+body
Content-Type	Media type của body	application/json
Serialization	Object → JSON	News response
Deserialization	JSON → Object	@RequestBody News
Idempotent	Lặp request có cùng hiệu ứng dự kiến	PUT/DELETE
CRUD	Create/Read/Update/Delete	POST/GET/PUT/DELETE
DispatcherServlet	Front controller của Spring MVC	dispatch request
HttpMessageConverter	Chuyển body ↔ object	Jackson JSON converter
Location header	URI resource mới sau create	/api/news/4

VIII. RESEARCH BASE VÀ AI BASE LEARNING

Quy tắc: AI được dùng để đặt giả thuyết, giải thích và gợi ý debug; sinh viên phải kiểm chứng bằng source chính thức, Postman, log và khả năng giải thích. Với nội dung có thể thay đổi theo phiên bản Spring, ưu tiên Spring Framework/Spring Boot documentation.

VIII.A. 10 prompt nghiên cứu chuyên sâu kỹ thuật

1. Phân tích @Controller và @RestController từ góc nhìn response body và view resolution.
2. Giải thích pipeline DispatcherServlet → handler mapping → argument resolver → message converter → response.
3. Giải thích cách @RequestBody sử dụng HttpMessageConverter/Jackson để deserialize JSON.
4. Phân tích @PathVariable và @RequestParam: semantics REST, type conversion và lỗi thường gặp.
5. Phân tích @ResponseStatus và ResponseEntity: khi nào dùng mỗi cách?
6. Giải thích cách @ExceptionHandler được chọn khi nhiều exception handler cùng tồn tại.
7. Phân tích @ControllerAdvice và @RestControllerAdvice.
8. Giải thích idempotency của PUT và DELETE; phân biệt với “response giống nhau”.
9. Phân tích content negotiation và Content-Type/Accept trong Spring MVC.

10. Giải thích vì sao constructor injection được ưu tiên cho Controller/Service/Repository.

VIII.B. 10 prompt thực tiễn production

1. Thiết kế chuẩn error response cho Spring Boot API gồm code, message, fieldErrors, traceId và timestamp.
2. Khi nào nên dùng DTO thay vì domain/entity trong REST API?
3. Làm sao thêm validation cho @RequestBody và chuẩn hóa lỗi validation?
4. Thiết kế pagination/sorting/filtering cho GET /api/news theo REST semantics.
5. Tổ chức package theo layer hay feature trong Spring Boot dự án lớn: trade-off?
6. Làm sao viết controller test bằng MockMvc mà không cần chạy server thật?
7. Làm sao thiết kế service transaction boundary khi chuyển từ Map sang JPA?
8. Làm sao xử lý optimistic locking/concurrent update cho PUT trong production?
9. Làm sao version API mà giữ backward compatibility?
10. Làm sao thêm authentication/authorization mà vẫn giữ Controller mỏng?

VIII.C. 10 prompt cập nhật công nghệ / xu hướng

1. Kiểm tra Spring Boot/Spring Framework hiện hành: thay đổi nào ảnh hưởng cách viết REST controller?
2. So sánh Spring MVC và Spring WebFlux cho REST API; khi nào không nên dùng reactive?
3. So sánh Spring Boot với Quarkus/Micronaut cho REST microservices.
4. Phân tích Java records làm DTO trong Spring Boot hiện đại.
5. OpenAPI 3 + springdoc-openapi đang được dùng thế nào với Spring Boot?
6. Problem Details for HTTP APIs (RFC 9457) ảnh hưởng thiết kế error response thế nào?
7. So sánh REST, GraphQL và gRPC cho hệ thống doanh nghiệp.
8. Native image/GraalVM ảnh hưởng startup/memory của Spring Boot API như thế nào?
9. Observability (metrics, tracing, logs) cho REST API Spring Boot hiện nay gồm những gì?
10. AI coding assistants có thể hỗ trợ controller/service test ra sao và cần guardrail gì?

VIII.D. AI Verification Worksheet

Trường	Sinh viên ghi
Prompt	
Claim/câu trả lời AI	
Nguồn kiểm chứng	Spring docs / log / Postman / code
Thử nghiệm đã chạy	
Kết quả	
Điều chỉnh cuối	
Tôi có thể tự giải thích?	Có / Chưa

IX. SINH VIÊN CẦN TỰ KIỂM TRA GÌ SAU BUỔI HỌC

IX.1. Knowledge check

1. @RestController khác @Controller như thế nào?
2. Tại sao không cần @ResponseBody trên từng method của @RestController?
3. @PathVariable và @RequestParam khác nhau về nghĩa API?
4. @RequestBody chuyển JSON thành object qua thành phần nào?
5. Khi nào dùng @ResponseStatus? Khi nào ResponseEntity linh hoạt hơn?
6. 3-layer gồm các layer nào và responsibility mỗi layer?
7. Tại sao business logic không nên đặt trong Controller?
8. POST tạo resource nên trả status gì? Location header có ý nghĩa gì?

9. PUT có idempotent không? Giải thích bằng state server.
10. Tại sao resource không tồn tại nên là 404 thay vì 500?
11. @ExceptionHandler khác @ControllerAdvice ở phạm vi thể nào?
12. Vì sao dữ liệu in-memory mất sau restart và điều đó có chấp nhận ở Slot 13 không?

IX.2. Skill check

Tôi có thể...	Đạt	Chưa đạt	Cần làm lại
Tạo Spring Boot project Java 21 + Spring Web			
Viết @RestController + base @RequestMapping			
Viết GET/POST/PUT/DELETE endpoint			
Bind path/query/body đúng annotation			
Tổ chức Controller → Service → Repository			
Trả 201/204/404 đúng tình huống			
Tạo global exception handler			
Test toàn bộ CRUD bằng Postman			
Trace GET và POST end-to-end			
Giải thích code mà không đọc thuộc lòng			

IX.3. Ba mức tự đánh giá

Mức	Dấu hiệu
Mức 1 – Chưa đạt	Chưa map được endpoint, annotation lẫn lộn, CRUD chưa chạy, không giải thích request flow.
Mức 2 – Đạt cơ bản	CRUD chạy; dùng đúng annotation; 3-layer cơ bản; test được Postman.
Mức 3 – Tốt	Status/error chuẩn; architecture rõ; trace request; sửa được transfer task; giải thích trade-off.

IX.4. Exit Ticket

- Vẽ lại request flow của POST /api/news bằng tối đa 7 hộp.
- Viết một lỗi bạn đã gặp và bằng chứng giúp tìm ra nguyên nhân.
- Nêu một điểm bạn sẽ thay đổi khi repository chuyển từ Map sang JPA.

PHỤ LỤC A. QUICK REFERENCE – ANNOTATIONS & HTTP

Need	Use first	Question to ask
REST controller	@RestController	Return value có đi vào response body?
Base path	@RequestMapping	Các endpoint có chung prefix không?
Read collection/item	@GetMapping	Path nào biểu diễn resource?
Create	@PostMapping	Body gì? status 201? Location?
Update	@PutMapping	Item có tồn tại? body đủ field?
Delete	@DeleteMapping	Sau xóa có body không?
Read URI id	@PathVariable	Đây là identity của resource?
Filter/search	@RequestParam	Đây là tùy chọn truy vấn?
Read JSON	@RequestBody	Content-Type có application/json?

Fixed status	@ResponseStatus	Status có luôn cố định?
Flexible response	ResponseEntity	Cần status/header/body động?
Handle exception	@ExceptionHandler	Exception nào → status nào?
Global handlers	@ControllerAdvice	Nhiều controller dùng chung?

PHỤ LỤC B. POSTMAN VERIFICATION MATRIX

ID	Request	Expected status	Body/Header evidence	Pass
T01	GET /api/hello	200	JSON + application/json	
T02	GET /api/hello?name=Student	200	message chứa Student	
T03	GET /api/news	200	JSON array	
T04	GET /api/news/1	200	News id 1	
T05	GET /api/news/999	404	ApiError	
T06	POST /api/news	201	Location + created body	
T07	PUT /api/news/2	200	updated body	
T08	PUT /api/news/999	404	ApiError	
T09	GET /api/news?keyword=spring	200	filtered array	
T10	DELETE /api/news/2	204	no body	
T11	GET deleted id	404	ApiError	
T12	POST wrong Content-Type	4xx	phân tích response/log	

PHỤ LỤC C. HUMAN VERIFICATION GATE & EVIDENCE PACKAGE

C.1. Human Verification Gate – không qua nếu chỉ “code chạy”

- Chỉ vào @RestController và giải thích điều gì xảy ra với return object.
- Mở một @GetMapping và nói đầy đủ HTTP method + URI cuối cùng.
- Chỉ vào @PathVariable và nói nó lấy dữ liệu từ đâu.
- Chỉ vào @RequestBody và giải thích JSON → object.
- Giải thích tại sao @RequestParam phù hợp với search.
- Trace POST /api/news từ Postman tới Map.
- Giải thích vì sao create trả 201 thay vì 200.
- Giải thích ý nghĩa Location header.
- Chỉ nơi NotFound được phát hiện và nơi nó được đổi thành 404.
- Giải thích tại sao Controller không gọi Map trực tiếp.
- Giải thích Service có business logic gì trong demo.
- Giải thích Repository sẽ thay đổi thể nào khi học JPA.
- Tạo một request gây 404 rồi giải thích response.
- Tạo một request gây 405 hoặc 415 rồi chẩn đoán.
- Sửa search từ keyword thành title rồi cập nhật Postman test.

C.2. Evidence Package

Evidence	Nội dung
E1	Ảnh project structure
E2	Ảnh application startup thành công
E3	Postman GET list + GET by id

E4	Postman POST 201 + Location
E5	Postman PUT 200
E6	Postman DELETE 204
E7	Postman 404 error body
E8	Request-flow trace worksheet
E9	Debug log ít nhất 1 lỗi
E10	Git log các checkpoint V0–V6 nếu yêu cầu

Kết nối sang Slot tiếp theo: Slot 13 kết thúc khi sinh viên hiểu và triển khai REST CRUD in-memory đúng semantics. Các chủ đề OpenAPI/Swagger, paging/sorting/testing và JPA sẽ mở rộng dần; không “nhảy cóc” sang database trước khi request/response và 3-layer được hiểu chắc.